ADITYA UNIVERSITY

**SCHOOL OF ENGINEERING**

**Department of Data Science**

# CERTIFICATE

This is to certify that the Cornerstone project work entitled *"***BUS TICKET BOOKING SYSTEM***"* is submitted by

| | |
|---|---|
| **V.D.A. ANUSHA** | **(24B11DS220)** |
| **K.S.R. VIKRANTH** | **(24B11DS232)** |
| **P. YESHWANTH REDDY** | **(24B11DS163)** |
| **P.H.SANDEEP KUMAR** | **(24B11DS158)** |

in partial fulfillment of the requirements for the award of the B.Tech. degree in Information Technology for the academic year 2025-2026.

**Project Guide**
Dr.Karnam Sreenu, Ph.D,
Assistant Professor,
Department of DS

**Head of the Department**
Dr. M. Rajababu, Ph.D,
Associate Professor,
Department of DS

# DECLARATION

We hereby declare that the Cornerstone project entitled "**BUS TICKET BOOKING SYSTEM**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Data science**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

**By**


V.D.A.ANUSHA (24B11DS220)
K.S.R.VIKRANTH (24B11DS232)
P.YESHWANTH REDDY (24B11DS163)
P.SANDEEP KUMAR (24B11DS158)

# ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr.Karnam Sreenu, Ph.D, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. A. Lakshmana Rao, Associate Professor and Mrs. S.Neelima, Assistant Professor**, Department of IT, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. M. Rajababu, Associate Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

**Table of contents**


| CONTENTS | PAGE NO |
|---|---|
| **ABSTRACT** | i |
| **LIST OF FIGURES** | ii |
| **LIST OF TABLES** | iii |
| **ACRONYMS** | iv |
| **1. INTRODUCTION** | 1 |
| 1.1 Brief Information about the Project | 1 |
| 1.2Motivation & contribution of project | 1 |
| 1.3 Objectives of the Project | 2 |
| 1.4 Scope of the Project | 2 |
| **2. SYSTEM ANALYSIS** | 3 |
| 2.1 Functional Requirements | 3 |
| 2.2 Non-Functional Requirements | 4 |
| 2.3 Requirements Specification | 5 |
| 2.3.1 Minimum Hardware Requirements | 5 |
| 2.3.2 Software Requirements | 5 |
| **3. TECHNOLOGY DESCRIPTION** | 6 |
| 3.1 Programming Language | 6 |
| 3.1.1 Introduction to Java | 6 |
| 3.2 Framework / Libraries | 6 |
| 3.2.1 Introduction to Console_based | 6 |
| 3.3 Database Technology | 7 |
| 3.3.1 JDBC (JavaDatabase | 7 |

Connectivity)

3.3.2 Oracle Database 8

3.4 Tools & IDEs 8

3.4.1 Visual Studio Code 8

**4. DATABASE DESIGN** 9

4.1 Database Tables 9

4.2 Table Structure 9

4.3 Keys and Constraints 10

4.4 Table Relationships 11

**5. SYSTEM DESIGN** 12

5.1 System Architecture 12

5.2 Interface Implementation 12

5.2.1 Bus management module 12

5.2.2 Booking module 13

5.2.3 Search & Availability module 13

5.2.4 Database connectivity module 13

5.2.5 Console interface module 13

**6. SYSTEM IMPLEMENTATION** 14

6.1 User Interface screen 14

6.2 Registration screen 15

6.3 Booking functionality 15

6.4 Ticket booking 16

6.5 Viewing ticket 16

6.6 Cancelling ticket

**7. CONCLUSION** 17

**8. FUTURE ENHANCEMENTS** 18

**9. REFERENCES** 19

**APPENDIX** (Sample Code) 20

**APPENDIX (SQL Queries)**

# ABSTRACT

The **Bus Ticket Booking System** is a console-based application developed to automate the process of reserving bus tickets efficiently. The system allows users to book tickets, cancel reservations, and check seat availability in real time through a menu-driven interface, ensuring smooth ticketing operations and preventing double bookings. It maintains organized information about bus routes and seat allocations using a relational database, enabling operators to manage reservations effectively.

Developed using **Core Java** with a **console interface**, **JDBC** for database connectivity, and **Oracle Database** for backend storage, the application ensures accurate data management, reduces errors, and maintains consistent records. The system follows a modular design, separating functionalities such as booking, cancellation, seat management, and database operations, which enhances maintainability and scalability.

With its menu-based interface, efficient data handling, and validation mechanisms, the **Bus Ticket Booking System** provides a practical and reliable solution for managing bus ticket reservations, improving operational efficiency and passenger convenience even in a console environment.

# LIST OF FIGURES


| Figure no. | Figure Title | Page no. |
|---|---|---|
| 1 | Login screen of Bus ticket booking system | 14 |
| 2 | Main Menu screen of bus ticket booking system | 14 |
| 3 | Ticket Booking Screen | 15 |
| 4 | Seat Availability Screen | 15 |
| 5 | Ticket Cancellation Screen | 16 |
| 6 | Bus Route Management Screen | 16 |
| 7 | Database Update Confirmation Screen | 16 |

# LIST OF TABLES


| Table no. | Table Title | Page no. |
|---|---|---|
| 1 | User | 9 |
| 2 | Bus | 9 |
| 3 | Ticket | 9 |

# ACRONYMS

- **BTBS** : Bus Ticket Booking System
- **JDBC** : Java Database Connectivity
- **UI** : User Interface
- **SQL** : Structured Query language

# 1. INTRODUCTION

## 1.1 Brief Information about the project

The Bus Ticket Booking System is a Java-based desktop application. It automates the process of reserving bus tickets efficiently. Users can book tickets, cancel reservations, and check seat availability in real time.

The system prevents double booking and ensures accurate ticket allocation. It maintains information about buses, routes, seats, customers, and bookings in database.

JDBC is used for database connectivity with Oracle Database as backend. The system follows a modular design for easy maintenance and scalability. It reduces manual effort, errors, and improves operational efficiency. This project demonstrates practical use of Java and database management in a real-world system.

## 1.2 Motivation and Contribution of Project

In many bus operators or travel agencies, ticket bookings are still handled manually or using simple spreadsheets, which leads to inefficiencies such as double bookings, difficulty in tracking seat availability, delayed updates, and increased chances of errors or data loss. These limitations motivated the development of the Bus Ticket Booking System.

The key contributions of this project include:

- Providing a digital solution for booking and managing bus tickets
- Reducing manual effort and paperwork for operators
- Preventing double bookings and improving accuracy of seat allocation
- Enabling real-time seat availability checks and quick ticket management
- Enhancing data security through controlled access and record management

This system simplifies ticket reservation and cancellation, improves and provides a reliable solution for passengers and bus operators.

## 1.3 Objectives of the Project

The main objectives of the Bus Ticket Booking System are:

- To develop a user-friendly application for managing bus ticket reservations
- To store bus, route, seat, customer, and booking information in a structured database
- To provide functionalities for booking, cancelling, and updating tickets
- To enable quick checking of seat availability and passenger details
- To display ticket and booking information in an organized and readable format

## 1.4 Scope of the Project

The Bus Ticket Booking System can be used by bus operators, travel agencies, and transportation companies to manage ticket reservations efficiently. It supports operations such as booking tickets, cancelling reservations, updating passenger details, checking seat availability, and displaying all bookings in an organized manner. The system provides a simple and user-friendly interface for managing ticketing and bus route information in a centralized database, ensuring accuracy and reducing errors.

# 2. SYSTEM ANALYSIS

## 2.1 Functional Requirements

Functional requirements define the core operations and functionalities that the Bus Ticket Booking System must perform to manage ticket reservations and bus operations efficiently. The functional requirements of the Bus Ticket Booking System include:

- **BookTickets**
  The system should allow users to book tickets for a selected bus, choosing seats and entering passenger details. Proper validation should ensure accurate booking and prevent double booking.

- **Cancel Tickets**
  The system should allow authorized users or passengers to cancel booked tickets. A confirmation mechanism should prevent accidental cancellations.

- **Check Seat Availability**
  The system should display available seats for a selected bus and date, enabling users to make informed booking decisions.

- **Search Booking Records**
  The system should support searching for bookings using specific criteria such as booking ID, customer name, or bus ID, returning accurate results quickly.

- **Display Bus and Booking Records**
  The system should display all buses, routes, and booking details in a structured format for easy reference. Sorting and filtering options can be included for better usability.

- **Manage Bus and Route Details**
  The system should allow the management of bus and route information, including adding, updating, and deleting bus and route records. Bookings should be associated with the respective bus and route.

## 2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the
Bus Ticket Booking System must satisfy.

- **Performance**
  The system should respond quickly to operations like booking, cancellation, seat availability checks, and searching without delays.
- **Security**
  Only authorized users (admins or staff) should access and modify data, using proper authentication and access control.
- **Usability**
  The system should have a simple and user-friendly interface, allowing users to perform tasks like booking or cancelling tickets without difficulty.
- **Reliability**
  The system should operate consistently and handle errors gracefully, ensuring no data loss during bookings or cancellations.

- **Scalability**
  The system should handle increasing numbers of buses, routes, bookings, and users without affecting performance.

## 2.3 Requirements Specification

This section specifies the hardware and software needed to run the **Bus Ticket Booking System**.

### 2.3.1 Minimum Hardware Requirements

- **Processor:** Intel i3 or above
- **RAM:** 4 GB (minimum)
- **Hard Disk:** 20 GB free space
- **Monitor:** Standard display
- **Keyboard and Mouse**

### 2.3.1 Software Requirements

- **Operating System:** Windows 7/10/11
- **Programming Language:** Java
- **IDE:** VS Code
- **Database:** Oracle Database
- **Connectivity:** JDBC

# 3. TECHNOLOGY DESCRIPTION

## 3.1 Programming language

### 3.1.1 Introduction to java

Java is a high-level, object-oriented programming language developed by Sun Microsystems. It is widely used for developing desktop, web, and enterprise applications due to its platform independence and robustness.

In the Bus Ticket Booking System, Java is used to implement the core functionality of the application. It handles operations such as booking tickets, cancelling reservations, updating passenger details, checking seat availability, and managing bus and route information. Java provides features like object-oriented programming, exception handling, and strong memory management, which help in building reliable and efficient applications.

One of the main advantages of Java is that it is platform-independent, meaning the program can run on any system that has a Java Virtual Machine (JVM). This makes it suitable for developing portable applications like the Bus Ticket Booking System.

## 3.2 Framework / Libraries

The Bus Ticket Booking System uses a console-based interface for user interaction. Users interact with the system by entering commands or selecting options from the menu displayed in the console. The system allows users to perform operations such as:

- Logging in
- Booking a ticket
- Cancelling a ticket
- Checking seat availability
- Displaying bookings and bus information

The console-based interface is simple, lightweight, and easy to use, allowing efficient interaction without the need for a graphical interface.

## 3.3 Database Technology

The Bus Ticket Booking System uses a relational database for storing and managing bus, route, seat, passenger, and booking information in a structured manner. The database ensures efficient data storage, retrieval, and manipulation, which is essential for maintaining accuracy and consistency in the system.

In this project, Oracle Database is used as the backend, and JDBC (Java Database Connectivity) is used to connect the Java application to the database, enabling operations such as inserting new bookings, updating seat status, cancelling tickets, and fetching booking details.

### 3.3.1 JDBC (Java Database Connectivity)

JDBC is an API (Application Programming Interface) in Java used to connect and interact with databases. It allows Java applications to execute SQL queries and retrieve or update data in the database.

In the Bus Ticket Booking System, JDBC is used to connect the Java application with the Oracle Database. It performs operations such as:

- Establishing a connection to the database
- Executing SQL queries (INSERT, UPDATE, DELETE, SELECT) for ticket bookings, cancellations, and seat updates
- Retrieving data about buses, routes, passengers, and bookings
- Closing the database connection

JDBC acts as a bridge between the application and the database, ensuring smooth communication, maintaining data consistency, and enabling efficient data handling for ticket reservations.

### 3.3.2 Oracle Database

Oracle Database is a powerful relational database management system used for storing and managing structured data. It provides high performance, reliability, and security for handling large volumes of data.

In the Bus Ticket Booking System, Oracle Database is used to store bus, route, seat, passenger, and booking details. It supports SQL for data manipulation and ensures data integrity through constraints such as primary keys, foreign keys, and proper data types. The database enables organized management of ticket reservations and real-time seat availability.

## 3.4 Tools & IDEs

### Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for the Bus Ticket Booking System. It is a lightweight code editor that supports Java programming through extensions.

VS Code provides features such as syntax highlighting, code completion, and debugging support, which help improve coding efficiency. Its simple and user-friendly interface makes it suitable for developing, testing, and maintaining the Bus Ticket Booking System application

# 4. DATABASE DESIGN

## 4.1 Database Tables

The Bus Ticket Booking System uses a relational database to store information in a structured manner. The database consists of two main tables: Bus and Booking.

- The Bus table stores details of different buses and routes, including bus ID, route ID, total seats, and departure time.
- The Booking table stores ticket reservation information and is linked to the Bus table. It includes booking ID, passenger details, seat number, and booking status.

.

## 4.2 Table Structure

### User Table

| Field Name | Data Type | Description | Constraint |
|---|---|---|---|
| user_id | NUMBER | User ID | Primary Key |
| username | VARCHAR2(100) | User Name | NOT NULL |
| password | VARCHAR2(50) | User Password | NOT NULL |

**Table 1: User Relational Database Table**

### Bus table

| Field Name | Data Type | Description | Constraint |
|---|---|---|---|
| Bus_id | NUMBER | Bus ID | Primary key |
| Source | VARCHAR(50) | Starting location | Not Null |
| Destination | VARCHAR(50) | Ending location | Not Null |
| Total_seats | NUMBER | Total no.of seats | Not Null |
| Seats_available | NUMBER | Available seats | Not Null |
| price | NUMBER | Ticket price | CHECK (PRICE >0) |

**Table 2: Bus Relational Database Table**

**Ticket Table**

| Field Name | Data Type | Description | Constraint |
|---|---|---|---|
| Ticket_id | NUMBER | Ticket ID | Primary key |
| User_id | NUMBER | User ID | Foreign key,NOT NULL |
| Bus_id | NUMBER | Bus ID | Foreign key,NOT NULL |
| Passenger_id | VARCHAR(100) | Name of Passenger | NOT NULL |
| Seat_number | NUMBER | Booked seat passenger | NOT NULL |
| Travel_date | DATE | Date of travel | NOT NULL |

**Table 3: Booking Relational Database Table**

## 4.3 Keys and Contraints

Keys and constraints are used to maintain data integrity and ensure accurate data storage.

- **Primary Keys:**
    - **User_id** in user table
    - **bus_id** in Bus table
    - **ticket_id** in ticket table
- **Foreign Key:**
    - **bus_id** , **user_id**
- **NOT NULL Constraint:**
    - Important fields such as **User_name, Password, passenger_name** , **seat_number** , **travel_date** , **source, destination, total_seats, seats_available** and cannot be left Empty

- **Data Type Constraints:**
    - Each field accepts only valid data types to ensure correctness (e.g., NUMBER for IDs, VARCHAR2 for names and status)

These constraints help maintain data integrity, avoid duplication, and ensure that only valid and consistent data is stored in the database.

## 4.4 Table Relationships

The **User**, **Bus**, and **Ticket** tables are related to manage bookings efficiently:

- **User Table** stores information about customers.
- **Bus Table** stores details about buses such as source, destination, total seats, and price.
- **Ticket Table** stores individual bookings made by users and references both the user and the bus.

**Relationships:**

- Each **ticket** is associated with **one user** and **one bus**.
- One **user** can book multiple tickets.
- One **bus** can have multiple tickets booked.

This represents **one-to-many relationships**:

- **User → Ticket** (one user can have many tickets)
- **Bus → Ticket** (one bus can have many tickets)

The **ticket table** acts as a junction table connecting users and buses, ensuring organized and consistent booking data.

# 5. SYSTEM DESIGN

## 5.1 System Architecture

The **Bus Ticket Booking System** follows a **three-layer architecture**:

1. **Presentation Layer (Console Interface)**
   - User interacts with the system through **console menus and prompts**
   - Operations include booking tickets, cancelling reservations, and viewing seat availability
2. **Application Layer (Business Logic)**
   - Processes user inputs from the console
   - Performs operations such as **book ticket, cancel ticket, update seat status, and check availability**
3. **Data Layer (Database)**
   - Stores **bus, route, seat, and booking data**
   - Managed using **Oracle Database**

   This architecture ensures **organized flow of operations, data integrity, and maintainability**.

## 5.2 Interface Implementation

The system is divided into the following modules:

1. **Bus Management Module**
   - Updates total seats and schedule
   - Stores bus and route details

**2. Booking Module**

- Handles ticket reservations via console prompts
- Cancels bookings and updates seat status

2. **Search & Availability Module**
    - Allows searching buses by route or bus ID
    - Displays available seats in real time on console

3. **Database Connectivity Module**
    - Connects the Java application with Oracle DB using JDBC
    - Executes SQL queries for booking, cancellation, and seat updates

4. **Console Interface Module**
    - Provides menu-driven interface in console
    - Handles user interactions like booking, cancelling, and viewing tickets

# 6.SYSTEM IMPLEMENTATION

## 6.1 User Interface Screen

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\Desktop\JAVA Programs>javac BusBookingSystem.jav

C:\Users\Lenovo\Desktop\JAVA Programs>java BusBookingSystem
===== BUS BOOKING SYSTEM =====

1. User Login
2. Register
3. Admin Login
4. Exit
Choose:
```

### 6.2 Registration Screen

```
1. User Login
2. Register
3. Admin Login
4. Exit
Choose: 2
Enter username: alice
Enter password: 123
Registration Successful!

1. User Login
2. Register
3. Admin Login
4. Exit
Choose: |
```

## 6.3 Booking Functionality

```
1. Search Bus
2. Book Ticket
3. View Tickets
4. Cancel Ticket
5. Logout
Choose: 1
Enter Source: Tallapudi
Enter Destination: Nellore
Bus ID: 6 | Seats: 20 | Price: 120
```

## 6.4 Ticket Booing

```
1. Search Bus
2. Book Ticket
3. View Tickets
4. Cancel Ticket
5. Logout
Choose: 2
Enter Bus ID: 6
Available Seats: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
Choose seat: 4
Passenger Name: alice
Payment Processing.......
Payment SUCESSFUL
Ticket Booked!
```

### 6.5 Viewing Tickets

```
1. Search Bus
2. Book Ticket
3. View Tickets
4. Cancel Ticket
5. Logout
Choose: 3
Ticket ID: 13 | Bus: 6 | Passenger: alice | Seat:
```

### 6.6 Cancelling Tickets

```
1. Search Bus
2. Book Ticket
3. View Tickets
4. Cancel Ticket
5. Logout
Choose: 4
Enter Ticket ID: 13
Ticket Cancelled!
```

# 7.CONCLUSION

The system has been successfully developed to manage bus ticket reservations in an efficient and accurate manner. It replaces traditional manual ticketing processes with a computerized solution, thereby minimizing human errors, saving time, and booking improving overall reliability. The application offers key functionalities such as ticket, reservation cancellation, seat status updates, and real-time seat availability, all implemented through a structured, menu-driven console interface.

By utilizing technologies such as Java, JDBC, and Oracle Database, the system ensures robust performance, secure data management, and seamless access to booking information. It effectively prevents issues like double booking while significantly reducing the workload of ticketing staff.

Overall, the system enhances operational efficiency and provides a scalable foundation for future improvements. Additional features such as web-based access, automated notifications, and detailed booking reports can be integrated to further extend its capabilities. Thus, the project successfully meets its objectives by delivering a reliable, organized, and efficient bus reservation management solution.

# 8. FUTURE ENHANCEMENTS

Some possible future enhancements include:

- **Online Booking and Web-Based Access:**
  Converting the console-based system into a web or mobile application, allowing users to book tickets remotely.

- **Multi-User Support:**
  Allowing multiple users to access the system simultaneously for booking and cancellations without conflicts.

- **Automated Notifications:**
  Sending SMS or email notifications to passengers about booking confirmations, cancellations, or schedule changes.

- **Dynamic Pricing and Discounts:**
  Implementing features for special fares, discounts, and offers based on routes or timings.

- **Reporting and Analytics:**
  Generating reports for total bookings, cancellations, seat occupancy, and revenue for better management.

These enhancements can make the system more scalable, user-friendly, and suitable for real-time use in bus ticket management.

# 9.REFERENCES

The following references were used during the development of the Employee Management System:


1. **Herbert Schildt**, *Java: The Complete Reference*, Oracle Press, 11th Edition.
2. **E. Balagurusamy**, *Programming with Java*, McGraw Hill Education.
3. **Korth, Silberschatz, Sudarshan**, *Database System Concepts*, McGraw Hill.
4. **Oracle Documentation**,
   https://docs.oracle.com
5. **Java Documentation (Oracle)**,
   https://docs.oracle.com/javase
6. **JDBC Tutorial – GeeksforGeeks**,
   https://www.geeksforgeeks.org/jdbc/
7. **Java Swing Tutorial – TutorialsPoint**,
   https://www.tutorialspoint.com/java_swing
8. **W3Schools Java Tutorial**,
   https://www.w3schools.com/java/

# APPENDIX

## A.1 Sample Code

## Database Connection Code:

The following code establishes a connection between the console-based Java application and the Oracle database using JDBC. It allows the application to perform operations such as inserting booking details, updating seat status, cancelling tickets, and retrieving data directly through the console interface.

**Database Connection (DBConnection.java)**

This class establishes a connection between the console-based Java application and the Oracle database using JDBC. It provides a reusable method getConnection() that returns a Connection object, allowing other modules to execute SQL queries for booking, cancelling, updating, and retrieving ticket and seat information.

```
import java.sql.*;

import java.util.*;

public class BusBookingSystem        {

static Scanner sc = new Scanner(System.in);

static int loggedInUserId = -1;

public static void main(String[] args) {

System.out.println("===== BUS BOOKING SYSTEM =====");

while (true) {

System.out.println("\n1. User Login");

System.out.println("2. Register");
```

```
System.out.println("3. Admin Login");
System.out.println("4. Exit");
System.out.print("Choose: ");
int choice = sc.nextInt();
switch (choice) {
case 1:
if (login()) userMenu();
else System.out.println("Invalid credentials!");
break;
case 2:
registerUser();
break;
case 3:
adminLogin();
break;
21
case 4:
System.out.println("Thank you!");
return;
default:
System.out.println("Invalid choice!");
}
}
```

```
//================= USER MENU =================
static void userMenu() {
System.out.println("===== USER MENU =====");
while (true) {
System.out.println("\n1. Search Bus");
System.out.println("2. Book Ticket");
System.out.println("3. View Tickets");
System.out.println("4. Cancel Ticket");
System.out.println("5. Logout");
System.out.print("Choose: ");
int choice = sc.nextInt();
switch (choice) {
case 1: searchBus(); break;
case 2: bookTicket(); break;
case 3: viewTickets(); break;
case 4: cancelTicket(); break;
case 5: System.out.println("Session Completed");
System.out.println("Logout Sucessful");
return;
default: System.out.println("Invalid choice!");
}
}
```

```
// ================= LOGIN =================
static boolean login() {
try {
Connection con = DBConnection.getConnection();
sc.nextLine();
System.out.print("Username: ");
String username = sc.nextLine();
System.out.print("Password: ");
String password = sc.nextLine();
String query = "SELECT user_id FROM users_table WHERE username=? AND
                    password=?";
PreparedStatement ps = con.prepareStatement(query);
ps.setString(1, username);
ps.setString(2, password);
ResultSet rs = ps.executeQuery();
if (rs.next()) {
loggedInUserId = rs.getInt("user_id");
return true;
}
} catch (Exception e) {
e.printStackTrace();
}
return false;
}
```

```
// ================= REGISTER =================
static void registerUser() {
try {
Connection con = DBConnection.getConnection();
System.out.print("Enter username: ");
sc.nextLine();
String username = sc.nextLine();
System.out.print("Enter password: ");
String password = sc.nextLine();
String idQuery = "SELECT NVL(MAX(user_id),0)+1 FROM users_table";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(idQuery);
int id = 1;
if (rs.next()) id = rs.getInt(1);
String query = "INSERT INTO users_table VALUES (?, ?, ?)";
PreparedStatement ps = con.prepareStatement(query);
ps.setInt(1, id);
ps.setString(2, username);
ps.setString(3, password);
ps.executeUpdate();
System.out.println("Registration Successful!");
} catch (Exception e) {
System.out.println("Username already exists!");
}
```

```
}
// ================= SEARCH BUS =================
static void searchBus() {
try {
Connection con = DBConnection.getConnection();
sc.nextLine();
System.out.print("Enter Source: ");
String source = sc.nextLine();
System.out.print("Enter Destination: ");
String dest = sc.nextLine();
String query = "SELECT * FROM buses_table WHERE source=? AND
                destination=?";
PreparedStatement ps = con.prepareStatement(query);
ps.setString(1, source);
ps.setString(2, dest);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
System.out.println("Bus ID: " + rs.getInt("bus_id") +
" | Seats: " + rs.getInt("seats_available") +
" | Price: " + rs.getDouble("price"));
}
} catch (Exception e) {
e.printStackTrace();
}
```

```
// ================= BOOK =================
static void bookTicket() {
try {
Connection con = DBConnection.getConnection();
System.out.print("Enter Bus ID: ");
int busId = sc.nextInt();
List<Integer> seats = getAvailableSeats(busId);
if (seats.isEmpty()) {
System.out.println("No seats available!");
return;
}
System.out.println("Available Seats: " + seats);
System.out.print("Choose seat: ");
int seat = sc.nextInt();
if (!seats.contains(seat)) {
System.out.println("Invalid seat!");
return;
}
System.out.print("Passenger Name: ");
sc.nextLine();
String name = sc.nextLine();
System.out.print("Enter Travel Date (YYYY-MM-DD): ");
String dateInput = sc.nextLine().trim();
```

```
java.sql.Date travelDate;
System.out.println("DEBUG INPUT = [" + dateInput + "]");
try {
travelDate = java.sql.Date.valueOf(dateInput);
} catch (Exception e) {
System.out.println("Invalid date format! Use YYYY-MM-DD.");
return;
}
String insert = "INSERT INTO tickets VALUES (ticket_seq.NEXTVAL, ?, ?, ?,
                ?,?)";
PreparedStatement ps = con.prepareStatement(insert);
ps.setInt(1, loggedInUserId);
ps.setInt(2, busId);
ps.setString(3, name);
ps.setInt(4, seat);
ps.setDate(5, travelDate);
ps.executeUpdate
String update = "UPDATE buses_table SET seats_available=seats_available-1
                WHERE bus_id=?";
PreparedStatement ps2 = con.prepareStatement(update);
ps2.setInt(1, busId);
ps2.executeUpdate();
System.out.println("Payment Processing.....");
System.out.println("Payment SUCESSFUL");
```

```
System.out.println("Ticket Booked!");
} catch (SQLException e) {
if (e.getErrorCode() == 1)
System.out.println("Seat already booked!");
else e.printStackTrace();
}
}
```

## SQL QUERIES

```
-- Table 1: User Table
CREATE TABLE users (
user_id NUMBER PRIMARY KEY,
username VARCHAR2(100) NOT NULL,
password VARCHAR2(50) NOT NULL
);

-- Table 2: Bus Table
CREATE TABLE bus (
bus_id NUMBER PRIMARY KEY,
source VARCHAR2(50) NOT NULL,
destination VARCHAR2(50) NOT NULL,
total_seats NUMBER NOT NULL,
seats_available NUMBER NOT NULL,
```

```
price NUMBER CHECK (price > 0)
);
-- Table 3: Ticket/Booking Table
CREATE TABLE ticket (
ticket_id NUMBER PRIMARY KEY,
user_id NUMBER NOT NULL,
bus_id NUMBER NOT NULL,
passenger_name VARCHAR2(100) NOT NULL,
seat_number NUMBER NOT NULL,
travel_date DATE NOT NULL,
CONSTRAINT fk_user FOREIGN KEY (user_id) REFERENCES users(user_id),
CONSTRAINT fk_bus FOREIGN KEY (bus_id) REFERENCES bus(bus_id)
);
```